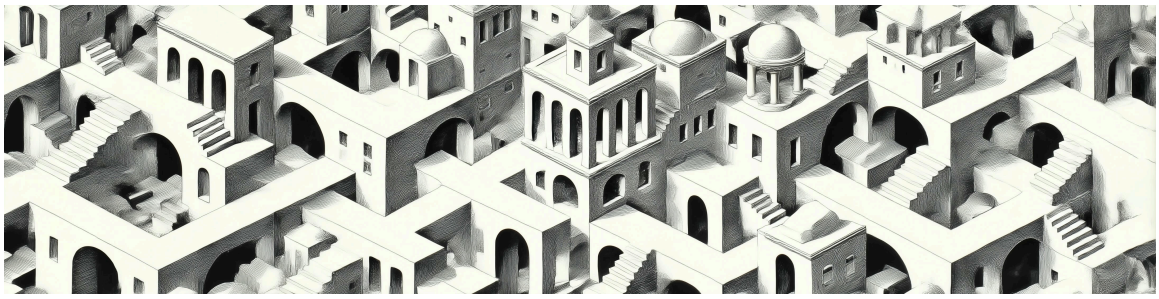
Algorithmen

Joern Ploennigs · AI4SC



> Everything should be made as simple as possible, but not simpler.
>
> — Albert Einstein

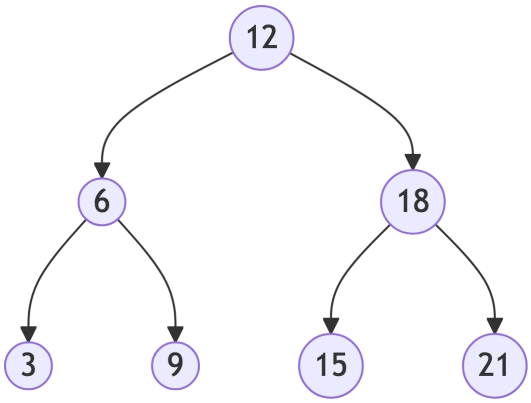
Problemklassen und Algorithmen

Heuristiken

Datenstrukturen: Bäume

Komplexe Datenstruktur – Bäume

Eine weitere wichtige komplexe (zusammengesetzte) Datenstruktur sind Bäume. Sie werden insbesondere für effizientes Suchen benutzt.



Wir stellen den Baum als Dictionaries dar, bei denen wir den Wert semantisch als `wert` und die Unterbäume mit `links` und `rechts` bezeichnen.

```
baum = {
        "wert": 12,
        "links":{
                "wert": 6,
                "links": {"wert":  3},
                "rechts":{"wert":  9}
        },
        "rechts":{
                "wert": 18,
                "links": {"wert": 15},
                "rechts":{"wert": 21}
        }
}
```

Wenn wir den Baum durchlaufen wollen, ist eine rekursive Funktion oft besonders naheliegend. Alternativ ist auch eine iterative Umsetzung mit einem expliziten Stack oder einer Queue moeglich. Die rekursive Funktion arbeitet auf dem aktuellen Knoten und ruft sich dann mit den Teilbaeumen der Kinder auf.

```
def traverse(tree):
    wert=tree["wert"]                # der aktuelle Wert
    print(wert)                      # gebe den aktuellen Wert aus
    if "links" in tree:              # wenn der unterbaum links definiert ist
        traverse(tree["links"])      # rufe die Funktion rekursiv auf
    if "rechts" in tree:             # wenn der unterbaum rechts definiert
ist
        traverse(tree["rechts"])     # rufe die Funktion rekursiv auf
```

```
traverse(baum)
```

```
12
6
3
9
18
15
21
```

Wenn wir den Baum rekursiv durchlaufen, sehen wir die Reihenfolge von oben nach unten und von links nach rechts. Wir beginnen beim Wurzelknoten 12, gehen dann links zur 6, weiter zur 3 und danach zur 9.

Sortieren

Sortier-Algorithmen

Bubble Sort

Das Sortieren von Elementen in einer Liste ist eine übliche Aufgabe beim Programmieren. Hierfür gibt es einige Standard-Algorithmen, die man kennen sollte.

Bubble Sort ist einer der einfachsten Sortieralgorithmen. Hierbei wird jedes Element der Liste durchlaufen und mit dem nächsten Element verglichen. Ist das zweite Element kleiner als das erste wird die Position getauscht. Die Liste wird so lange wieder und wieder durchlaufen bis dieser Fall nicht mehr auftritt.

```
def bubbleSort(numbers):
    for i in range(len(numbers)-1):
        for j in range(0, len(numbers)-i-1):
            if numbers[j] > numbers[j + 1] :
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
```

```
zahlenfolge = [12, 6, 3, 9, 18, 15, 21]

bubbleSort(zahlenfolge)

zahlenfolge
```

```
[3, 6, 9, 12, 15, 18, 21]
```

Es ist zu beachten, dass Bubble Sort die Liste direkt modifiziert, also die vorherige Reihenfolge geändert wird.

QuickSort

QuickSort ist ein Divide-and-Conquer-Algorithmus. Er waehlt ein Element aus der Liste als zentrales Element und erzeugt dann jeweils eine Liste der Elemente, die kleiner als das zentrale Element sind, eine Liste mit gleichen Elementen und eine Liste mit groesseren Elementen. Diese Teillisten werden dann wieder mit QuickSort sortiert, bis ihre Groesse 1 erreicht. So wird das Sortierproblem sukzessive immer kleiner.

```
def quickSort(elements):
    less = []
    equal = []
    greater = []

    if len(elements) > 1:
        pivot = elements[len(elements) // 2]
        for x in elements:
```

```
            if x < pivot:
                less.append(x)
            elif x == pivot:
                equal.append(x)
            elif x > pivot:
                greater.append(x)
        return quickSort(less) + equal + quickSort(greater)
    else:  # Falls nur ein oder kein Element in der list ist
        return elements
```

Das Ergebnis ist eine sortierte Liste welche neu ist und zugewiesen werden muss.

```
zahlenfolge = [12, 6, 3, 9, 18, 15, 21]

zahlenfolge=quickSort(zahlenfolge)

zahlenfolge
```

```
[3, 6, 9, 12, 15, 18, 21]
```

Insertion Sort

Ein weiterer Algorithmus zum Sortieren ist Insertion Sort. Er basiert auf der menschlichen Strategie, Karten zu sortieren, indem Werte an der passenden Stelle eingefuegt werden.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and key < arr[j] :
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
```

```
zahlenfolge = [12, 6, 3, 9, 18, 15, 21]

insertion_sort(zahlenfolge)

zahlenfolge
```

```
[3, 6, 9, 12, 15, 18, 21]
```

Python Standardfunktion `sorted`

Die Lösung im Programmieralltag ist die in Python eingebaute Funktion `sorted()`. Hier wird die Liste nicht modifiziert und muss neu zugewiesen werden.

```
zahlenfolge = [12, 6, 3, 9, 18, 15, 21]

zahlenfolge=sorted(zahlenfolge)

zahlenfolge
```

```
[3, 6, 9, 12, 15, 18, 21]
```

Performancevergleich der Sortierfunktionen

Im Performancevergleich der Funktionen schneidet Bubble Sort schlechter ab als Insertion Sort. Die eingebaute Funktion `sorted()` ist deutlich schneller als beide. Sie nutzt mit Timsort einen hybriden Algorithmus, der Ideen aus Merge Sort und Insertion Sort kombiniert, und ist in C implementiert.

```
import timeit
timeit.timeit(lambda: bubbleSort([12, 6, 3, 9, 18, 15, 21]), number=30)
```

```
2.554204547777772e-05
```

```
timeit.timeit(lambda: quickSort([12, 6, 3, 9, 18, 15, 21]), number=30)
```

```
3.654096508398652e-05
```

```
timeit.timeit(lambda: insertion_sort([12, 6, 3, 9, 18, 15, 21]), number=30)
```

```
1.1499971151351929e-05
```

```
timeit.timeit(lambda: sorted([12, 6, 3, 9, 18, 15, 21]), number=30)
```

```
5.709007382392883e-06
```

```
zahlenfolge_gross = list(range(10000))
zahlenfolge_gross.reverse()
```

```
timeit.timeit(lambda: bubbleSort(zahlenfolge_gross.copy()), number=30)
```

```
71.35452937503578
```

```
timeit.timeit(lambda: quickSort(zahlenfolge_gross.copy()), number=30)
```

```
0.09486375004053116
```

```
timeit.timeit(lambda: insertion_sort(zahlenfolge_gross.copy()), number=30)
```

```
43.87245545798214
```

```
timeit.timeit(lambda: sorted(zahlenfolge_gross.copy()), number=30)
```

```
0.0022850829991512
```

Hier zeigt sich erneut, dass BubbleSort am schlechtesten abscheidet aufgrund der vielen Vergleiche. Quick Sort schneidet besser ab, weil es das Problem in kleinere Probleme unterteilt. Insertion Sort ist allerdings immer noch performanter. Am schnellsten löst das Problem die eingebaute `sorted()` Funktion, welche auch auf dem Insertion Sort Algorithmus basiert.

Suchen

Suchen von Elementen in Listen

Vollständige Suche

Der naive Ansatz ist die vollständige Suche, indem wir in einer For-Each-Schleife durch alle Elemente in einer Liste iterieren, um zu sehen ob das Element irgendwo zu finden ist.

```
def contains(list, x):
    for l in list:
        if(l == x):
            return True
    return False
```

```
zahlenfolge = [12, 6, 3, 9, 18, 15, 21]

contains(zahlenfolge, 15)
```

```
True
```

```
contains(zahlenfolge, 1)
```

```
False
```

Suchen mit binaerer Suche

Die binaere Suche ist ein Divide-and-Conquer-Algorithmus zum Suchen in einer *sortierten* Liste. Sie betrachtet immer das mittlere Element und entscheidet dann, ob das gesuchte Element links oder rechts weitergesucht werden muss.

```
def binary_search_sub(arr, suchwert, low, high):
    while low <= high:
        mid = low + (high - low)//2
        if arr[mid] == suchwert:
            return mid
        elif arr[mid] > suchwert:
            return binary_search_sub(arr, suchwert, low, mid - 1)
        else:
            return binary_search_sub(arr,  suchwert, mid + 1, high)
    return -1

def binary_search(arr_sorted, suchwert):
    return binary_search_sub(arr_sorted, suchwert, 0, len(arr_sorted) - 1)
```

```
zahlenfolge_sortiert = sorted(zahlenfolge)
binary_search(zahlenfolge_sortiert, 15)
```

```
4
```

```
binary_search(zahlenfolge_sortiert, 1)
```

```
-1
```

Suchen mit binären Suchbäumen

Der am Anfang aufgestellte Datenstruktur Baum stellt einen binären Suchbaum dar und kann zur Suche verwendet werden. Wir nutzen hierbei aus, dass dieser *bereits sortiert* ist.

```
def suchBaum(tree, suchwert):
    wert=tree["wert"]
    if wert == suchwert:
        return True
    if suchwert < wert and "links" in tree:
        return suchBaum(tree["links"], suchwert)
    elif suchwert > wert and "rechts" in tree:
        return suchBaum(tree["rechts"], suchwert)
    else:
        return False
```

```
suchBaum(baum, 15)
```

```
True
```

```
suchBaum(baum, 1)
```

```
False
```

Binäre Suchbäume erstellen

Bei der Nutzung eines binären Suchbaums kann man auch beim Einfügen ausnutzen, dass er immer sortiert ist. Insbesondere bei wachsenden Datenstrukturen die sortiert vorliegen müssen, macht das Suchbäume äußerst effektiv.

```
def erweitereBaum(tree, neuer_wert):
    wert=tree["wert"]
    if wert == neuer_wert:
        return
    elif neuer_wert < wert:
        if "links" not in tree:
            tree["links"]={"wert":neuer_wert}
        else:
            return erweitereBaum(tree["links"], neuer_wert)
    else:
        if "rechts" not in tree:
            tree["rechts"]={"wert":neuer_wert}
        else:
            return erweitereBaum(tree["rechts"], neuer_wert)
```

```
baum2 = {"wert": 12}
for wert in [12, 6, 3, 9, 18, 15, 21]:
    erweitereBaum(baum2, wert)
```

```
print(baum)
```

```
{'wert': 12, 'links': {'wert': 6, 'links': {'wert': 3}, 'rechts': {'wert':
9}}, 'rechts': {'wert': 18, 'links': {'wert': 15}, 'rechts': {'wert': 21}}}
```

```
print(baum2)
```

```
{'wert': 12, 'links': {'wert': 6, 'links': {'wert': 3}, 'rechts': {'wert':
9}}, 'rechts': {'wert': 18, 'links': {'wert': 15}, 'rechts': {'wert': 21}}}
```

Suchen mit Python in Listen und Sets

Im Alltag kann die in Python eingebaute Suche mit `15 in zahlenfolge` genutzt werden. Insbesondere bei grossen Datenmengen lohnt sich fuer reine Enthaltensein-Abfragen haeufig die Umwandlung in ein Set. Python-Sets sind hashbasiert und fuer die schnelle Mitgliedschaftspruefung optimiert.

```
15 in zahlenfolge
```

```
True
```

```
zahlenset = set(zahlenfolge)
15 in zahlenset
```

```
True
```

Performancevergleich der Suchfunktionen

```
timeit.timeit(lambda: contains(zahlenfolge, 15), number=30)
```

```
3.3339601941406727e-06
```

```
timeit.timeit(lambda: binary_search(zahlenfolge_sortiert, 15), number=30)
```

```
6.999995093792677e-06
```

```
timeit.timeit(lambda: suchBaum(baum, 15), number=30)
```

```
4.749977961182594e-06
```

```
timeit.timeit(lambda: 15 in zahlenfolge, number=30)
```

```
1.7500133253633976e-06
```

```
zahlenset = set(zahlenfolge)
timeit.timeit(lambda: 15 in zahlenset, number=30)
```

```
1.2919772416353226e-06
```

```
zahlenfolge_gross = list(range(5000))
zahlenfolge_gross.reverse()
zahlenfolge_gross_sortiert = sorted(zahlenfolge_gross)
zahlenset_gross = set(zahlenfolge_gross)
```

```
baum_gross = {"wert": zahlenfolge_gross[len(zahlenfolge_gross)//2]}
step = len(zahlenfolge_gross)//4
while step>0:
    idx=step
    while idx < len(zahlenfolge_gross):
        erweitereBaum(baum_gross, zahlenfolge_gross[idx])
        idx=idx+step
    step = step // 2
```

```
timeit.timeit(lambda: contains(zahlenfolge_gross, 15), number=30)
```

```
0.0007704999879933894
```

```
timeit.timeit(lambda: binary_search(zahlenfolge_gross_sortiert, 15),
number=30)
```

```
2.3540982510894537e-05
```

```
timeit.timeit(lambda: suchBaum(baum_gross, 15), number=30)
```

```
2.0416046027094126e-05
```

```
timeit.timeit(lambda: 15 in zahlenfolge_gross, number=30)
```

```
0.0007422500057145953
```

```
timeit.timeit(lambda: 15 in zahlenset_gross, number=30)
```

```
9.59029421210289e-07
```

Das Ergebnis sieht hier deutlich anders aus. Die lineare Listenabfrage schneidet nun wesentlich schlechter ab, waehrend die binaere Suche auf der sortierten Liste deutlich besser skaliert. Der

Suchbaum bleibt ebenfalls effizient, weil die Ordnung der Werte bei der Suche direkt genutzt wird. Fuer reine Mitgliedschaftspruefungen ist das Set hier am schnellsten.

KI und Algorithmenauswahl

Quiz

Bibliographie